AMR Course Project

Sunesh Praveen Raja Sundarasami, Sai Mukkundan Ramamoorthy

***Abstract*—This project presents an integrated navigation framework for autonomous mobile robots, addressing the critical challenges of path planning, localization, and environment exploration on the Robile platform. We implement a hybrid path planning system combining an A* algorithm with a novel clearance-based heuristic that penalizes proximity to obstacles through weighted zones. The particle filter localization module employs Monte Carlo methods with dual motion models (linear and circular) and optimized sensor likelihood calculations using adaptive ray subsampling and ball tree spatial indexing. For autonomous exploration, we develop a frontier detection system using DBSCAN clustering with multi-criteria decision making that balances information gain, distance costs, and safety clearance threshold. Key contributions include a two-phase path simplification algorithm preserving critical turns through Douglas-Peucker filtering and angular analysis, potential field control with emergency obstacle avoidance mechanisms (0.4m safety radius), and adaptive particle filter optimizations for real-time performance. This work provides a validated framework for deploying theoretical navigation algorithms on resource-constrained robotic platforms, addressing the critical gap between simulation and real-world deployment. Project code is available at https://github.com/HBRS-AMR/amr-project-amr-t04.**

***Index Terms*—A-Star with modified heuristic path planning, Clustering based Exploration, Ray-Casting based Particle Filter**

I. Introduction

The development of autonomous mobile robots capable of navigating unknown environments has significant implications across multiple industries and research domains. Path planning allows robots to efficiently reach target destinations while avoiding obstacles, localization is essential for robots to accurately determine their position within an environment and autonomous exploration capabilities enable robots to map and navigate previously unknown environments, by expanding their operational range and utility.

This work focuses on implementing fundamental navigation capabilities on the Robile robot platform, addressing the critical challenges of path planning, localization, and environment exploration. In this work, we have procedurally built the Astar algorithm, particle filter and exploration algorithm from scratch and further modified and slowly scaled the algorithms to satisfy the real-time constraints on Robile platform.

A. Motivation

The following work is done to complete the AMR course and to tackle the challenges such as sensor noise, environmental variability, and computational constraints that are introduced while implementing navigation capabilities on a real-world robot platform.

B. Problem Statement

The deployment of navigation algorithms on physical robots introduces several challenges that are not present in simulation. The potential field method suffers from limitations including unreachable targets, local optima, and difficulties with dynamic obstacles. These issues become more pronounced in physical implementations due to sensor noise and actuation uncertainties. Localization in real environments faces challenges of sensor limitations and environmental variability, where the position of a moving robot can drift significantly from the planned path over time. Environment exploration requires effective strategies for selecting exploration targets at the frontiers, while simultaneously building an accurate map through SLAM techniques. This dual process must operate efficiently within the computational constraints of the robot platform. The reliable and seamless integration of the above components on a physical robot platform represents the core challenge of this project.

C. Proposed Approach

This project proposes a comprehensive approach to autonomous robot navigation by integrating complementary methods for path planning, localization, and exploration on the Robile platform.

- For path planning, we will implement a hybrid approach that combines global path planning using the A* algorithm with custom heuristic using a clearance map with a potential field based motion planner. This combination allows the robot to plan efficient routes through large environments while maintaining reactive obstacle avoidance capabilities.
- For localization, we will implement Monte Carlo localization (particle filter) that enables the robot to estimate its position within an existing map. In this probabilistic approach, we handle sensor noise and environmental uncertainties inherent in real-world deployments using a velocity based motion model and a combination of gausian likelihood and raycasting based sensor model to accurately track the position of robile.
- For environment exploration, we will develop clustering based frontier detection algorithms that identify and select the exploration targets at the boundary between explored and unexplored regions and navigate towards it using above Astar based path planner while also integrating with SLAM techniques seamlessly to build and update the environmental map. This approach enables the robot to efficiently explore unknown environments.

*Submitted to the Department of Computer Science at Hochschule Bonn-Rhein-Sieg in partial fulfilment of the requirements for the degree of Master of Science in Autonomous Systems

†Supervised by Dr. Alex Mitrevski (Research and Strategy Manager, Lecturer/ Institute for AI and Autonomous Systems (A2S))

‡Submitted in March 2025

Throughout the project, we will evaluate the performance of these components individually and as an integrated system on the Robile platform, comparing results with simulation to identify and address real-world implementation challenges.

II. Related Work

PATH PLANNING Path planning is essential for autonomous robots, focusing on finding efficient routes from start to goal positions. Algorithms like A* and Dijkstra's are commonly used due to their effectiveness in static environments, though their performance can degrade in dynamic or complex settings. Enhancements, such as clearance-based heuristics and dynamic obstacle avoidance, improve safety and adaptability in real-world applications [1].

LOCALIZATION Accurate localization, crucial for autonomous navigation, is achieved using probabilistic methods like Monte Carlo Localization (MCL). MCL utilizes particle filters to estimate a robot's pose, handling sensor noise and uncertainties, making it ideal for real-time localization in dynamic environments [2].

III. Methodology

A. Path Planner

1) Overview: The navigation system implemented for the Robile platform consists of four main components working in concert to enable autonomous navigation.

- The Potential Field Controller serves as the core motion control system, generating attractive forces toward goals while creating repulsive forces from obstacles. It includes sophisticated features for emergency obstacle avoidance, local minima detection and escape mechanisms, and velocity smoothing with acceleration limits to ensure safe operation at maximum linear and angular velocities of 0.12 m/s and 0.15 rad/s respectively.
- Supporting this controller, the End Pose Publisher provides configurable goal positions at a 50Hz update rate, including both position and orientation information for comprehensive navigation targets.
- The Clearance Map Generator (Fig. 1) creates safety maps using distance transforms that account for both obstacles (80% weighting) and unexplored areas (20% weighting), applying Gaussian smoothing for gradual transitions that improve path planning safety.
- Finally, the Path Visualizer renders the planned paths in RViz using directional arrow markers with color gradients to indicate progress.

The system incorporates multiple safety features, including an emergency stop triggered at 0.4m from obstacles, hysteresis for stable recovery, and backup-and-rotate behaviors when the robot encounters challenging situations. Path planning combines A* algorithm with clearance-based optimization, while the potential field approach handles real-time motion control with distance and angle thresholds of 0.4 meters and 0.5 radians respectively. This integrated approach enables the Robile to navigate efficiently while maintaining safety in complex environments.

2) Workflow:

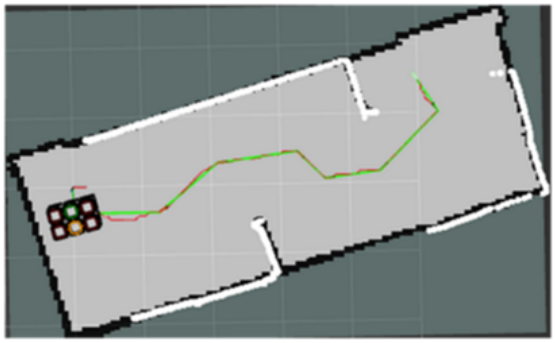


Fig. 1: Visualized path of Astar

a) A Path Planning:*

1) Graph Creation: The node first processes the incoming occupancy grid map. It applies an obstacle dilation function to thicken obstacles and then clears the robot's footprint from the map. With this processed (raw) map, the code creates a grid-based graph using NetworkX. In this graph:
   - Every free cell becomes a node.
   - Edges are added between adjacent nodes (including diagonal connections).
   - Nodes corresponding to obstacles (or unexplored areas) are removed, and each remaining node is assigned a weight inversely related to its clearance—computed using a clearance map (distance from obstacles).
2) Start and Goal Conversion:
   Before launching the A* search, the robot's current pose and the desired end pose are converted from world coordinates to grid (cell) coordinates using the map's resolution and origin.
3) Alternative Goal Handling:
   If the direct goal cell is not reachable (either not present in the graph or no path exists), the code searches around the target in expanding circles. It checks candidate points to see if they can connect to both the starting and target regions using the graph. The best candidate—based on a minimal distance from the desired target—is then used as an alternative endpoint for planning.
4) A* Execution:
   The core planning uses NetworkX's implementation of A*. It is invoked using the start and end grid coordinates on the weighted graph. A custom heuristic function is provided:
   - This heuristic computes a Euclidean distance between points and applies an exponential penalty if a node is near obstacles.
   - The node's clearance (obtained from the previously computed clearance map) is used to add additional cost. For example, if the clearance is very low (close to obstacles), a high penalty (like 100) is applied. This steers the search towards paths that maintain a safe distance from obstacles.

The heuristic function from the A* path planner is expressed as:

$$h(n) = d_{euclidean} \cdot \left(1 + 100 \cdot e^{-c/2}\right) + P_{obstacle} \tag{1}$$

where:

$$c_a(x, y) = \min_{(i,j) \in O} \sqrt{(x - i)^2 + (y - j)^2} \tag{2}$$

$$d_{euclidean} = \sqrt{(x_b - x_a)^2 + (y_b - y_a)^2} \tag{3}$$

$$P_{obstacle} = \begin{cases} 100 & \text{if } c < 2.0 \\ 50 & \text{if } 2.0 \leq c < 3.0 \\ 25 & \text{if } 3.0 \leq c < 4.0 \\ 0 & \text{if } c \geq 4.0 \end{cases} \tag{4}$$

Here:

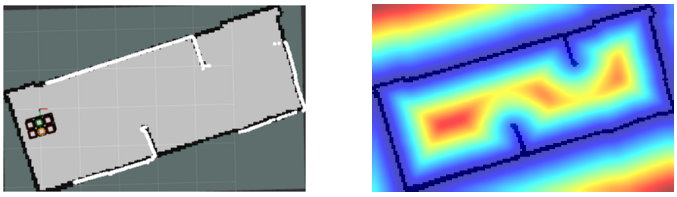
- $h(n)$ is the heuristic cost.
- c is the clearance value at position a.
- (x_a, y_a) is the current position.
- (x_b, y_b) is the goal position.
- $e^{-c/2}$ is the exponential decay factor for clearance.
- $P_{obstacle}$ is the progressive penalty based on clearance zones.

The complete cost function for A* is given by:

$$f(n) = g(n) + h(n) \tag{5}$$

where $g(n)$ is the actual cost from the start node to the current node.

5) Extraction of turns:
The extraction of turns is done by a two-phase path simplification algorithm that converts dense grid-based paths into efficient waypoint sequences while preserving critical turning points. The first phase uses the Douglas-Peucker simplification to reduce path complexity, retaining points that exceed a threshold distance (epsilon=2.0) from line segments, ensuring geometric accuracy. In the second phase, consecutive path segments are analyzed by calculating the angle between three consecutive points. Points are retained if the angle exceeds 30°, adjacent segments exceed 2 units, or the point represents a genuine directional change.
The algorithm computes stable angles and maintains consistent direction throughout the path. It enforces minimum segment lengths to prevent point clustering and ensures smooth transitions between waypoints. Finally, post-processing guarantees the inclusion of start and end points, validates segment lengths, and removes redundant waypoints. This approach efficiently simplifies paths for robotic motion planning while maintaining navigational accuracy.



(a) Occupancy grid (b) Clearance map

Fig. 2: (a) Occupancy grid and (b) Clearance map.

b) Potential Field Based Motion Planner:

- *Potential Field Controller Architecture* The Potential Field Controller represents a sophisticated real-time navigation solution that combines virtual force fields with reactive obstacle avoidance. At its core, the system creates an artificial potential field where the robot is simultaneously attracted to its goal while being repelled from obstacles, resulting in smooth and safe navigation through complex environments. The control system operates through a continuous feedback loop, processing real-time sensor data to maintain precise navigation control. When a new goal is received, the controller establishes an attractive force vector pointing toward the target position, with the magnitude of this force proportional to the distance from the goal. This ensures smooth deceleration as the robot approaches its destination.
- *Obstacle Avoidance Mechanism* The Obstacle Avoidance Mechanism is a key component of the controller, processing laser scan data to detect obstacles within a $\pm 60°$ forward arc. Each detected obstacle generates a repulsive force whose magnitude is inversely proportional to the distance, preventing collisions while ensuring smooth trajectory adjustments. The combination of attractive and repulsive forces creates a resultant force vector that guides the robot's motion. Additionally, the controller implements a sophisticated emergency handling system that activates when obstacles are detected within a critical safety radius of 0.4 meters. Upon activation, the system executes a three-phase recovery process: immediate stop, controlled backward motion, and strategic rotation to find a clear path. To prevent oscillation between normal and emergency states, a hysteresis mechanism requires a clearance of 0.3 meters before resuming normal operation.
- *Goal Approach Strategy* For precise goal achievement, the system employs a two-phase approach strategy. During the initial approach phase, the controller balances attractive and repulsive forces while maintaining a maximum linear velocity of 0.12 m/s. As the robot comes within 0.4 meters of the goal, it transitions to a precision control phase, focusing on accurate orientation alignment with reduced velocities.
- *Advanced Navigation Features* The controller also incorporates Advanced Navigation Features that enhance its robustness and adaptability. A local minima detection mechanism monitors the robot's progress, identifying situations where it might become trapped in a potential field well. When such scenarios arise, the controller initiates

an escape maneuver, generating a temporary movement perpendicular to the goal direction to break free. Velocity control is managed through sophisticated smoothing and scaling algorithms, ensuring smooth motion transitions with acceleration limits set at 0.2 m/s^2. Angular velocity is separately controlled with a maximum rate of 0.15 rad/s, enabling precise turning movements while maintaining stability.

- *ROS2 Integration* Finally, the system is deeply integrated with ROS2, maintaining continuous communication through various topics. Real-time debug information and status updates are published at 1 Hz, providing comprehensive monitoring of the robot's state, including current forces, velocities, and goal progress. A service interface allows external systems to query the controller's status, facilitating integration with higher-level planning systems.
- *Summary*
  - The code takes an occupancy grid, processes it (dilates obstacles, clears the robot's footprint, computes clearance), and converts it into a graph where cells are nodes and edges connect neighboring free cells. A* is performed on this graph using a clearance-aware heuristic to penalize low-clearance paths. If the goal is not directly reachable, an alternative goal is computed. Finally, the resulting path is simplified using geometric algorithms before being published for the robot to follow.
  - Through this comprehensive approach to navigation control, the Potential Field Controller achieves robust and efficient goal-seeking behavior while maintaining safety in dynamic environments.

B. Particle Filter Implementation

1) Overview: The particle filter is a probabilistic algorithm that estimates the robot's pose (position and orientation) by maintaining a set of weighted particles. Each particle represents a hypothesis of the robot's pose, and the weights are updated based on sensor data and a likelihood model.The implementation leverages ROS2 nodes, occupancy grids, and laser scan data to perform localization. Key components include particle initialization, motion updates, sensor updates, resampling, and occupancy field generation.

Particle filters represent a powerful probabilistic approach to state estimation. For robot localization, the key mathematical foundations include:

a) State-Space Model: The particle filter operates within a hidden Markov model where: The hidden state variables x_k represent the robot's pose (position and orientation) at time k. Observable variables y_k are sensor measurements (like laser scan data). A state transition model $p(x_k|x_{k-1})$ describes how the robot moves (motion model). An observation model $p(y_k|x_k)$ relates measurements to states (sensor model).

b) Recursive Bayesian Estimation: The core of the particle filter is Bayesian updating:

$$p(x_k|y_{1:k}) = \frac{p(y_k|x_k)p(x_k|y_{1:k-1})}{p(y_k|y_{1:k-1})} \tag{6}$$

Where the posterior distribution is approximated by a set of weighted particles.

c) Motion Models: Two primary motion models are implemented:

- Linear Motion Model For straight-line movement:

$$x_t = x_{t-1} + v_x \cos(\theta_{t-1})\Delta t - v_y \sin(\theta_{t-1})\Delta t + \epsilon_x \tag{7}$$

$$y_t = y_{t-1} + v_x \sin(\theta_{t-1})\Delta t + v_y \cos(\theta_{t-1})\Delta t + \epsilon_y \tag{8}$$

$$\theta_t = \theta_{t-1} + \omega\Delta t + \epsilon_\theta \tag{9}$$

- Circular Motion Model For curved trajectories when angular velocity is significant: graphicx

$$\begin{aligned} x_t = x_{t-1} &+ \frac{v_x}{\omega}\left(\sin(\theta_{t-1} + \omega\Delta t) - \sin(\theta_{t-1})\right) \\ &- \frac{v_y}{\omega}\left(\cos(\theta_{t-1} + \omega\Delta t) - \cos(\theta_{t-1})\right) \end{aligned} \tag{10}$$

$$\begin{aligned} y_t = y_{t-1} &+ \frac{v_x}{\omega}\left(\cos(\theta_{t-1}) - \cos(\theta_{t-1} + \omega\Delta t)\right) \\ &+ \frac{v_y}{\omega}\left(\sin(\theta_{t-1} + \omega\Delta t) - \sin(\theta_{t-1})\right) \end{aligned} \tag{11}$$

d) Sensor Models: **Likelihood Field Model**

$$p(z_t|x_t) = \prod_{i=1}^{n}[z_{hit} \cdot p_{hit}(z_t^i|x_t) + z_{rand} \cdot p_{rand}(z_t^i)] \tag{12}$$

Where:

- z_{hit} and z_{rand} are mixing parameters
- p_{hit} is a Gaussian probability centered at the expected measurement distance
- p_{rand} represents random measurements with uniform distribution

e) Ray Casting Model: An optimized approach that simulates expected laser readings for each particle and compares them to actual sensor readings.

f) Resampling Process: Particles are resampled with probability proportional to their weights:

$$P(\text{select particle}_i) = \frac{w_i}{\sum_{j=1}^{N} w_j} \tag{13}$$

2) Implementation Workflow:

1) **Initialization:** Particles are initialized around an initial pose estimate.Each particle is given equal initial weight.
2) **Motion Update:** Particles are propagated based on velocity commands.Noise is added proportional to velocity magnitude. Different models are used for linear vs. circular motion
3) **Sensor Update:** Laser scan data is transformed to the robot's frame. The likelihood of each particle is calculated. Weights are updated based on how well

predicted measurements match actual readings.Log-likelihood is used for numerical stability

4) **Resampling:** Particles are randomly sampled based on their weights.Low-weight particles are replaced.Random particles are injected to maintain diversity
5) **Pose Estimation:** The robot's pose is estimated as a weighted average of the best particles

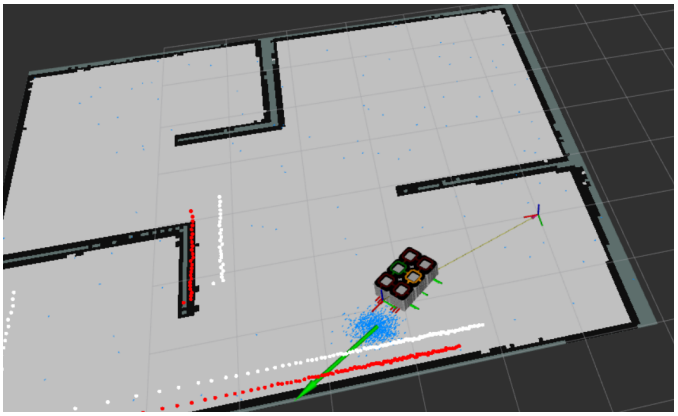


Fig. 3: Particle filter output simulation

3) Optimization Techniques:

1) **Efficient Distance Queries:** The ball tree algorithm provides $O(\log n)$ complexity for nearest neighbor lookups, significantly speeding up likelihood calculations.
2) **Adaptive Ray Subsampling:** This reduces computational load by using only a subset of laser beams, employing finer sampling near the robot and coarser sampling at longer ranges.
3) **Log-Likelihood Calculation:** This prevents numerical underflow when dealing with very small probabilities and improves numerical stability of weight calculations.
4) **Diversity Maintenance:** Random particle injection prevents sample impoverishment, orientation diversity is explicitly maintained, and boundary handling prevents particles from getting stuck at map edges.
5) **Adaptive Parameter Tuning:** Noise levels are adjusted based on robot velocity, and variance thresholds are used to prevent overconfidence in estimates.

4) Summary: This project implements a robust particle filter for robot localization using ROS2. The modular design, efficient occupancy field queries, and robust transform handling make it a strong foundation for localization tasks. Through the above techniques, the particle filter provides reliable localization for mobile robots in 2D environments, balancing computational efficiency with robustness to real-world uncertainties.

C. Autonomous Exploration System

1) Overview: The Autonomous Exploration System is a sophisticated framework designed for robotic navigation in unknown environments. It employs a frontier-based approach, integrating advanced algorithms for map analysis, decision-making, and execution within a ROS2 architecture. The system's core components include: Exploration Control System,Frontier Detection Methodology,Goal Selection Architecture,Progress Monitoring System,Visualization Framework
These components work in concert to enable efficient, safe, and adaptive autonomous exploration.

2) Workflow:

a) Frontier Detection: Processes occupancy grid maps to identify boundaries between known and unknown space.Employs neighborhood analysis considering direct and diagonal adjacent cells.Handles map edge conditions and out-of-bounds regions

b) Goal Selection: Utilizes DBSCAN clustering to identify significant exploration opportunities.Implements a multi-criteria(Eq. 14) decision-making process with weighted scoring. Evaluates goals based on information gain, distance costs, clearance, and historical attempts. Employs an adaptive blacklisting mechanism for problematic areas

$$S_{total} = w_u \cdot U + w_o \cdot C - w_d \cdot \frac{D}{D_{max}} \tag{14}$$

Where:

- S_{total} is the total score for a candidate point.
- U is the count of unexplored cells within a given radius.
- C is the clearance value $[0, 1]$.
- D is the distance to the robot.
- D_{max} is the maximum allowed distance.
- w_u, w_o, and w_o are weight parameters that balance the contributions of unexplored area, clearance, and distance.

c) Exploration Execution: Navigates to selected goals using ROS2 navigation stack. Continuously monitors progress and robot status

d) Progress Monitoring : Tracks the status of frontier cells within selected clusters. Computes exploration completion ratios. Triggers new goal selection when appropriate

e) Visualization and Feedback: Generates real-time feedback through RViz using marker arrays. Implements color-coding for different frontier states and cluster memberships. Provides dynamic updates of frontier clusters, robot position, and goal selection status.

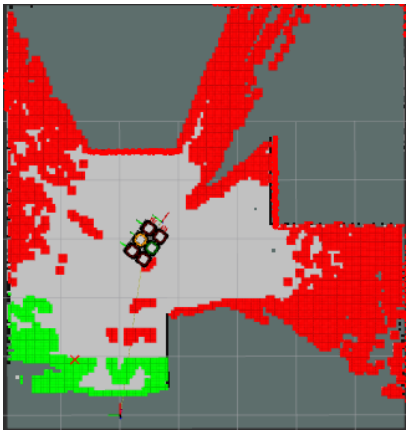


Fig. 4: Cluster based Explorer simulation output

f) Safety and Recovery: Incorporates multiple layers of safety checks. Detects and responds to stuck conditions. Manages timeouts and implements emergency stop capabilities.

g) Performance Optimization: Employs efficient numpy operations for map processing. Implements spatial indexing and clustering algorithms. Utilizes adaptive parameter tuning based on exploration progress

3) Summary: The Autonomous Exploration System presents a comprehensive solution for robotic exploration of unknown environments. Its modular architecture, built on ROS2, integrates sophisticated algorithms for frontier detection, goal selection, and progress monitoring. The system's adaptive behaviors and safety mechanisms ensure robust performance in various scenarios, while its visualization framework provides crucial real-time feedback. Through continuous optimization and intelligent decision-making, the system maintains a balance between exploration efficiency and thorough coverage, making it suitable for a wide range of autonomous navigation applications.

IV. Evaluation

The project successfully integrated path planning, localization, and exploration modules on the Robile platform, achieving safe navigation with a maximum velocity of 0.12 m/s. The A* path planner with clearance-based optimization effectively avoided obstacles while ensuring smooth paths. The particle filter maintained localization accuracy despite sensor noise, leveraging Gaussian and ray-casting models for robust state estimation. Exploration performance was enhanced through DBSCAN-based frontier detection and multi-criteria goal selection, ensuring efficient coverage of unknown environments. Key optimizations like adaptive ray subsampling and diversity-preserving resampling improved real-time performance. The evaluation highlights the system's robustness in handling real-world uncertainties, bridging the gap between simulation and deployment.

V. Conclusions

A. Summary

The project presents an integrated navigation framework for autonomous mobile robots on the Robile platform, addressing path planning, localization, and environment exploration. Path planning combines A* with a clearance-based heuristic to avoid obstacles while ensuring efficiency. Localization is handled through a particle filter with dual motion models and optimized sensor likelihood calculations. Exploration utilizes DBSCAN-based frontier detection, balancing information gain, distance, and safety. Key contributions include a two-phase path simplification algorithm, emergency obstacle avoidance, and adaptive particle filter optimizations. The project successfully integrates these components on physical hardware, achieving safe navigation and reliable localization while identifying future improvements for faster robots and more efficient clearance map generation.

B. Contributions

Sunesh Praveen Raja Sundarasami - Path and Motion Planning, Localisation, Environment Exploration.
Sai Mukkundan - Localisation, Report generation.

C. Future Work

In Path and Motion Planning - the generation of clearance map is time consuming, try to come up with more heuristic. Localisation - The method we used is not that effective for fast moving robot, the measurement and motion model can be calculated in a more efficient way to solve this.Additional sensory modalities such as depth cameras or inertial measurement units could enhance localization robustness in feature-poor environments. Finally, developing learning-based approaches for parameter adaptation could optimize performance across different environmental contexts, allowing the system to autonomously adjust exploration strategies and localization parameters.

References

[1] H. Qin, S. Shao, T. Wang, X. Yu, Y. Jiang, and Z. Cao, "Review of autonomous path planning algorithms for mobile robots," *Drones*, vol. 7, no. 3, 2023. [Online]. Available: https://www.mdpi.com/2504-446X/7/3/211

[2] D. Fox *et al.*, "Monte carlo localization: Efficient position estimation for mobile robots," *Proceedings of the National Conference on Artificial Intelligence*, pp. 343–349, 1999. [Online]. Available: https://robots.stanford.edu/papers/fox.aaai99.pdf

ACKNOWLEDGMENT

We acknowledge the support of teaching assistant Anudeep Sai Akula in setting up the robile platform and guiding us incase of any issues.

STATEMENT OF ORIGINALITY

We, the undersigned below, declare that this work has not previously been submitted to this or any other university and that it is, unless otherwise stated, entirely my own work. The report was, in part, written with the help of the AI assistant [Claude 3.7] as described in the appendix. We am aware that content generated by AI systems is no substitute for careful scientific work, which is why all AI-generated content has been critically reviewed by me, and We take full responsibility for it.

	Sunesh & Sai Mukkundan
Date	Signature

APPENDIX

1) Github Copilot was used for fixing errors.
2) Claude 3.7 Sonnet was used for grammatical corrections in report generation.

D. Videos

- **Localization:** Localization Video
- **Exploration and Navigation:** Exploration and Navigation Video

REFERENCES

[1] H. Qin, S. Shao, T. Wang, X. Yu, Y. Jiang, and Z. Cao, "Review of autonomous path planning algorithms for mobile robots," *Drones*, vol. 7, no. 3, 2023. [Online]. Available: https://www.mdpi.com/2504-446X/7/3/211

[2] D. Fox *et al.*, "Monte carlo localization: Efficient position estimation for mobile robots," *Proceedings of the National Conference on Artificial Intelligence*, pp. 343–349, 1999. [Online]. Available: https://robots.stanford.edu/papers/fox.aaai99.pdf